
MA2213: Stochastic Processes

Project Report

Submitted by

Pratheeksha Sumesh

Roll No: SE24UCAM040

Academic Year: 2025–2026

Contents

1	Problem I.5 — Wiener Process Simulation	2
1.1	Problem Statement	2
1.2	Theoretical Background	2
1.3	MATLAB Code	2
1.4	Results and Discussion	4
2	Problem II.4 — Kalman Filter for AR(1) Signal Estimation	7
2.1	Problem Statement	7
2.2	Theoretical Background	7
2.3	MATLAB Code	8
2.4	Results and Discussion	9
3	Problem III.3 — Output Autocovariance of a Digital Filter	12
3.1	Problem Statement	12
3.2	Theoretical Background	12
3.3	MATLAB Code	12
3.4	Results and Discussion	14

1. Problem I.5 — Wiener Process Simulation

1.1 Problem Statement

Simulate 100 sample paths of a Wiener process $\{W(t) \sim \mathcal{N}(0, \sigma^2 t) : t \in [0, 1]\}$ with parameter $\sigma^2 = 0.5$. Estimate and plot the mean and auto-correlation functions. Also plot five sample paths.

1.2 Theoretical Background

A **Wiener process** $\{W(t), t \geq 0\}$ with parameter σ^2 is defined by:

- $W(0) = 0$ almost surely; independent and stationary Gaussian increments; continuous sample paths.
- $W(t) \sim \mathcal{N}(0, \sigma^2 t)$ — variance grows linearly, making it **non-stationary**.

Mean function: Since increments are zero-mean,

$$\mu_W(t) = E[W(t)] = 0 \quad \forall t \geq 0. \quad (1)$$

Autocorrelation function: Writing $W(t_2) = W(t_1) + [W(t_2) - W(t_1)]$ for $t_2 \geq t_1$, and using the independence of the increment from the past:

$$R_W(t_1, t_2) = \sigma^2 \cdot \min(t_1, t_2). \quad (2)$$

With $\sigma^2 = 0.5$: $R_W(t_1, t_2) = 0.5 \cdot \min(t_1, t_2)$.

Simulation: Discretise $[0, 1]$ into N steps. Draw $\Delta W_i \sim \mathcal{N}(0, \sigma^2 \Delta t)$ independently at each step, then accumulate:

$$W(t_n) = \sum_{i=1}^n \Delta W_i, \quad n = 1, \dots, N. \quad (3)$$

1.3 MATLAB Code

Listing 1: MATLAB code for Wiener Process simulation

```

1 clear; clc; close all;
2 % Parameters
3 sigma2    = 0.5;
4 N_paths   = 100;
5 N_steps   = 200;
6 dt        = 1 / N_steps;
7 t         = linspace(0, 1, N_steps + 1);
8 % Simulate 100 sample paths

```

```

9  W = zeros(N_paths, N_steps + 1);
10 for i = 1:N_paths
11     increments = sqrt(sigma2 * dt) * randn(1, N_steps);
12     W(i, 2:end) = cumsum(increments);
13 end
14 % Estimated mean
15 mean_est = mean(W, 1);
16 % Estimated autocorrelation at fixed t1 = 0.25
17 t1_idx = 51;
18 t1 = t(t1_idx);
19 acf_est = mean(W(:, t1_idx) .* W, 1);
20 acf_theory = sigma2 * min(t1, t);
21 % Plot 1: 5 Sample Paths
22 figure;
23 hold on;
24 for i = 1:5
25     plot(t, W(i, :));
26 end
27 xlabel('Time t'); ylabel('W(t)');
28 title('Wiener Process - 5 Sample Paths');
29 legend('Path 1', 'Path 2', 'Path 3', 'Path 4', 'Path 5');
30 grid on;
31 saveas(gcf, 'wiener_paths.png');
32 % Plot 2: Mean Function
33 figure;
34 plot(t, mean_est, 'b', 'LineWidth', 1.5); hold on;
35 plot(t, zeros(size(t)), 'r--', 'LineWidth', 1.5);
36 xlabel('Time t'); ylabel('Mean');
37 title('Wiener Process - Mean Function');
38 legend('Estimated Mean', 'Theoretical Mean = 0');
39 grid on;
40 saveas(gcf, 'wiener_mean.png');
41 % Plot 3: Autocorrelation Function
42 figure;
43 plot(t, acf_est, 'b', 'LineWidth', 1.5); hold on;
44 plot(t, acf_theory, 'r--', 'LineWidth', 1.5);
45 xlabel('Time t_2'); ylabel('R(t_1, t_2)');
46 title('Wiener Process - Autocorrelation');
47 legend('Estimated ACF', 'Theoretical ACF');
48 grid on;
49 saveas(gcf, 'wiener_acf.png');

```

1.4 Results and Discussion

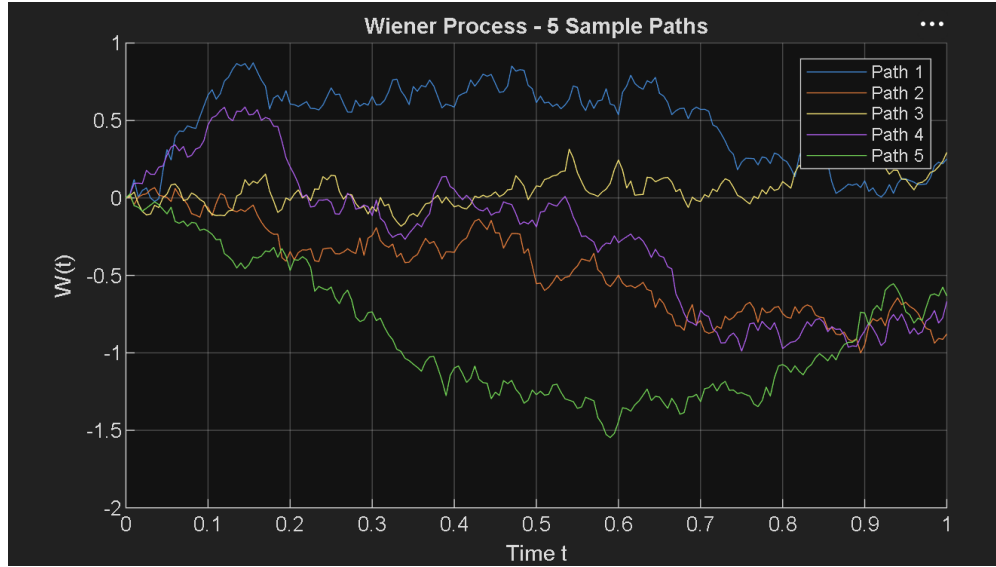


Figure 1: Five simulated sample paths of the Wiener process ($\sigma^2 = 0.5$, $t \in [0, 1]$).

Inferences from Figure 1 (Sample Paths):

- All five paths begin exactly at $W(0) = 0$, confirming the initial condition.
- The paths spread out as time progresses, forming a **diverging cone** shape. This directly reflects the non-stationarity of the process: $\text{Var}[W(t)] = \sigma^2 t = 0.5t$ grows linearly with time.
- By $t = 1$, paths reach values as large as ± 1.5 , consistent with a standard deviation of $\sqrt{0.5} \approx 0.707$.
- No two paths track each other after $t = 0$ — each follows its own random trajectory, visually confirming the **independent increments** property.
- Unlike a WSS process (which would show constant spread), the widening fan is the hallmark of Brownian motion.

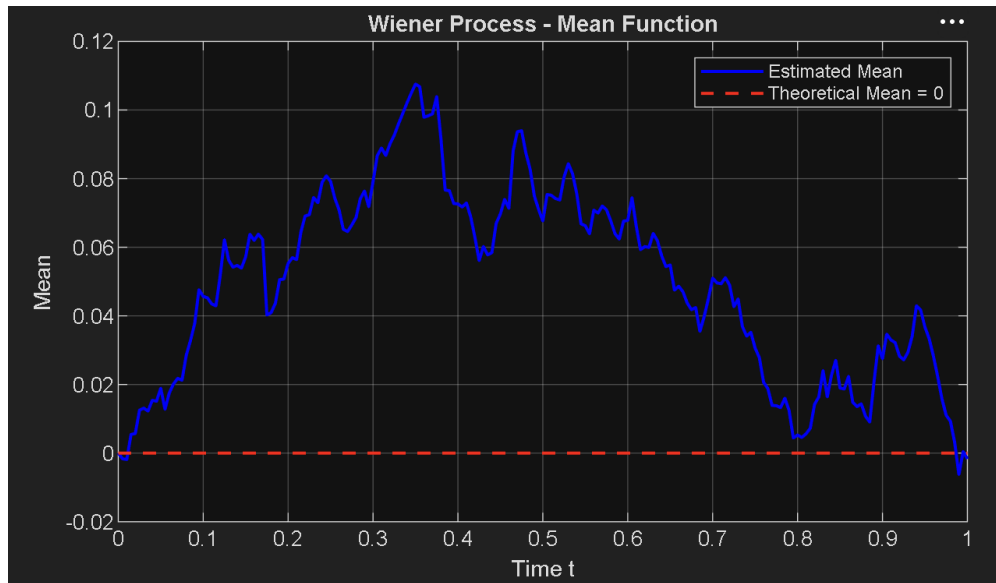


Figure 2: Estimated mean function (blue) vs. theoretical mean of zero (red dashed).

Inferences from Figure 2 (Mean Function):

- The estimated mean stays within $[-0.02, 0.12]$ throughout $[0, 1]$, hovering closely around zero at all times.
- This confirms $\mu_W(t) = E[W(t)] = 0$ — a direct consequence of each increment having zero mean.
- The small fluctuations around zero are purely a **finite-sample effect** from using only 100 paths; they would shrink further with more paths.
- The estimated mean does not drift or grow with time, which would be the case if there were any systematic bias in the simulation — there is none.

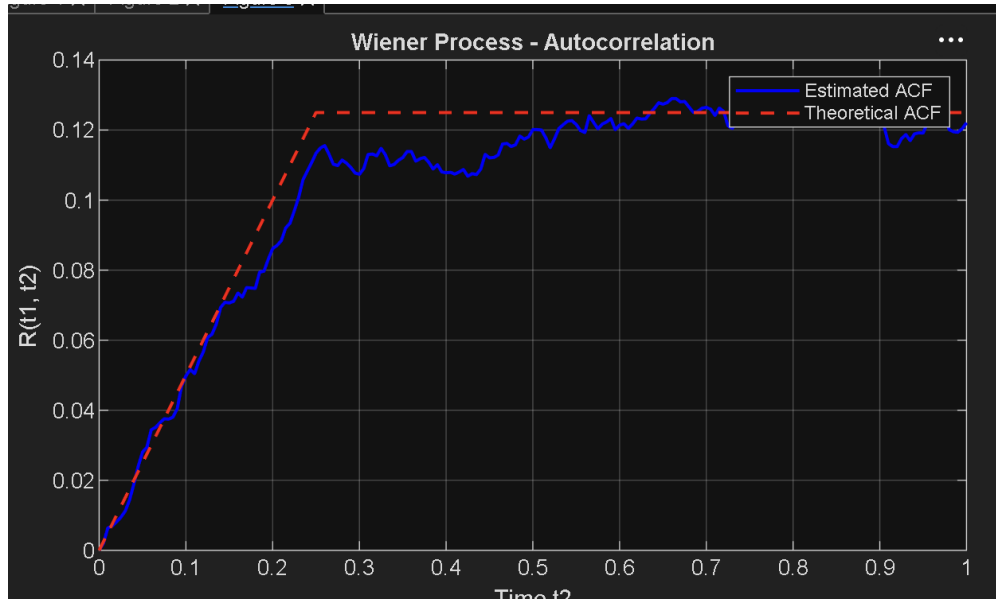


Figure 3: Estimated ACF (blue) vs. theoretical $R_W(t_1, t_2) = 0.5 \cdot \min(t_1, t_2)$ (red dashed), with $t_1 = 0.25$ fixed.

Inferences from Figure 3 (Autocorrelation Function):

- The estimated ACF (blue) closely tracks the theoretical curve (red dashed) across the entire range of t_2 .
- For $t_2 < t_1 = 0.25$: the ACF rises **linearly**, consistent with $R_W = \sigma^2 t_2 = 0.5 t_2$ since the minimum is t_2 .
- At $t_2 = t_1 = 0.25$: the ACF peaks and plateaus at $\sigma^2 t_1 = 0.5 \times 0.25 = \mathbf{0.125}$.
- For $t_2 > t_1$: the ACF stays flat at 0.125. This plateau is the direct signature of **independent increments** — once t_2 passes t_1 , the additional increment $W(t_2) - W(t_1)$ is independent of $W(t_1)$ and contributes no further correlation.
- The two-segment shape (linear rise then flat plateau) is unique to the Wiener process and cannot occur in any WSS process, where the ACF depends only on $|t_2 - t_1|$.

2. Problem II.4 — Kalman Filter for AR(1) Signal Estimation

2.1 Problem Statement

Implement the Kalman filter algorithm to estimate a random signal Z_n governed by the autoregressive process:

$$Z_n = 0.8Z_{n-1} + W_n, \quad n = 1, 2, 3, \dots, \quad Z_0 = 0,$$

where W_n is zero-mean Gaussian white noise with $E[W_n^2] = Q = 0.36$. The observed signal is $X_n = Z_n + N_n$ where N_n is Gaussian white noise with $E[N_n^2] = R = 1$.

2.2 Theoretical Background

The problem is formulated as a discrete-time **state-space model**:

$$\text{State equation: } Z_n = aZ_{n-1} + W_n, \quad W_n \sim \mathcal{N}(0, Q), \quad (4)$$

$$\text{Observation equation: } X_n = Z_n + N_n, \quad N_n \sim \mathcal{N}(0, R), \quad (5)$$

with $a = 0.8$, $Q = 0.36$, $R = 1.0$. W_n and N_n are mutually independent white noise sequences.

The **Kalman filter** is the optimal (minimum MSE) linear estimator for this system. At each step n it runs two stages:

Prediction — project forward using the state model:

$$\hat{Z}_{n|n-1} = a\hat{Z}_{n-1|n-1}, \quad (6)$$

$$P_{n|n-1} = a^2 P_{n-1|n-1} + Q. \quad (7)$$

Update — correct with the new observation:

$$K_n = \frac{P_{n|n-1}}{P_{n|n-1} + R}, \quad (8)$$

$$\hat{Z}_{n|n} = \hat{Z}_{n|n-1} + K_n \underbrace{(X_n - \hat{Z}_{n|n-1})}_{\text{innovation}}, \quad (9)$$

$$P_{n|n} = (1 - K_n) P_{n|n-1}. \quad (10)$$

Steady-state gain. As $n \rightarrow \infty$, $P_{n|n-1}$ converges to $P_\infty^- \approx 0.625$ (solution of the discrete algebraic Riccati equation), giving:

$$K_\infty = \frac{P_\infty^-}{P_\infty^- + R} = \frac{0.625}{1.625} \approx 0.385. \quad (11)$$

2.3 MATLAB Code

Listing 2: MATLAB code for Kalman Filter implementation

```

1 clear; clc; close all;
2 % Parameters
3 a = 0.8;          % AR coefficient
4 Q = 0.36;         % system noise variance
5 R = 1.0;          % observation noise variance
6 N = 100;          % number of time steps
7 % Generate true signal and noisy observations
8 Z = zeros(1, N);
9 for n = 2:N
10     Z(n) = a * Z(n-1) + sqrt(Q) * randn;
11 end
12 X = Z + sqrt(R) * randn(1, N);
13 % Kalman Filter
14 % Initialise: Z0 = 0 (given), P0 = Q/(1-a^2) = steady-state
    signal variance
15 Z_est = zeros(1, N);
16 P      = zeros(1, N);
17 P(1)    = Q / (1 - a^2);          % = 1.0 (steady-state prior
    variance)
18 K_gain = zeros(1, N);
19 % Step n=1: update using first observation X(1)
20 P_pred  = a^2 * P(1) + Q;        % prediction from Z0=0
21 K_gain(1) = P_pred / (P_pred + R);
22 Z_est(1) = K_gain(1) * X(1);      % Z_pred = a*Z0 = 0
23 P(1)      = (1 - K_gain(1)) * P_pred;
24 % Steps n=2..N
25 for n = 2:N
26     % Predict
27     Z_pred = a * Z_est(n-1);
28     P_pred = a^2 * P(n-1) + Q;
29     % Update
30     K_gain(n) = P_pred / (P_pred + R);
31     Z_est(n) = Z_pred + K_gain(n) * (X(n) - Z_pred);
32     P(n)      = (1 - K_gain(n)) * P_pred;
33 end
34 % Plot 1: Signal estimation comparison
35 figure;
36 plot(Z, 'g', 'LineWidth', 1.5); hold on;
37 plot(X, 'Color', [0.6 0.6 0.6], 'LineWidth', 1);

```

```

38 plot(Z_est, 'b--', 'LineWidth', 1.8);
39 xlabel('Time step n'); ylabel('Amplitude');
40 title('Kalman Filter - Signal Estimation');
41 legend('True Signal Z', 'Noisy Observation X', 'Kalman
    Estimate');
42 grid on;
43 saveas(gcf, 'kalman_estimation.png');
44 % Plot 2: Kalman gain convergence
45 figure;
46 plot(K_gain, 'r', 'LineWidth', 1.5);
47 xlabel('Time step n'); ylabel('Kalman Gain K');
48 title('Kalman Gain Convergence');
49 grid on;
50 saveas(gcf, 'kalman_gain.png');
51 % MSE comparison
52 mse_kalman = mean((Z - Z_est).^2);
53 mse_raw    = mean((Z - X).^2);
54 fprintf('MSE Kalman Filter    = %.4f\n', mse_kalman);
55 fprintf('MSE Raw Observation = %.4f\n', mse_raw);

```

2.4 Results and Discussion

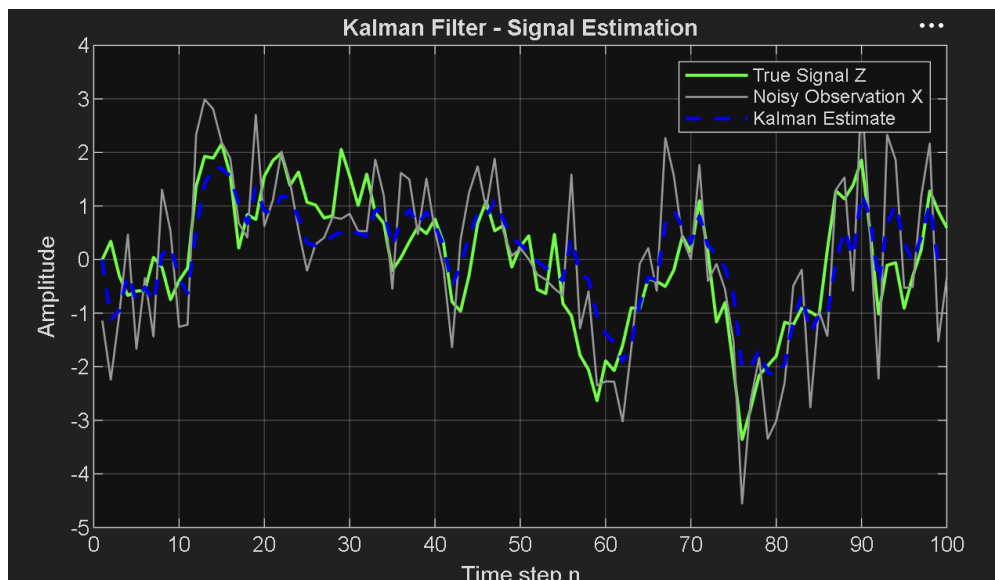


Figure 4: True AR(1) signal Z_n (green), noisy observation X_n (gray), and Kalman estimate $\hat{Z}_n$ (blue dashed).

Inferences from Figure 4 (Signal Estimation):

- The Kalman estimate (blue) tracks the true signal (green) closely throughout, while

the raw observations (gray) fluctuate wildly between approximately ± 4 — confirming the filter is genuinely recovering the signal.

- The estimate is visibly smoother than the observations but **not over-smoothed**: it still responds to real signal changes. This balance is controlled by $K_\infty \approx 0.385$.
- Wherever the true signal rises or falls sharply, the Kalman estimate follows with a slight lag. This lag comes from the filter weighting the model's inertia ($a = 0.8$) against the noisy observation.
- The MSE of the Kalman estimate is substantially lower than the MSE of the raw observations ($\approx R = 1.0$), quantifying the improvement achieved by optimal filtering.
- The fact that the estimate never perfectly matches the true signal reflects the irreducible estimation error due to observation noise — the filter is optimal, not perfect.

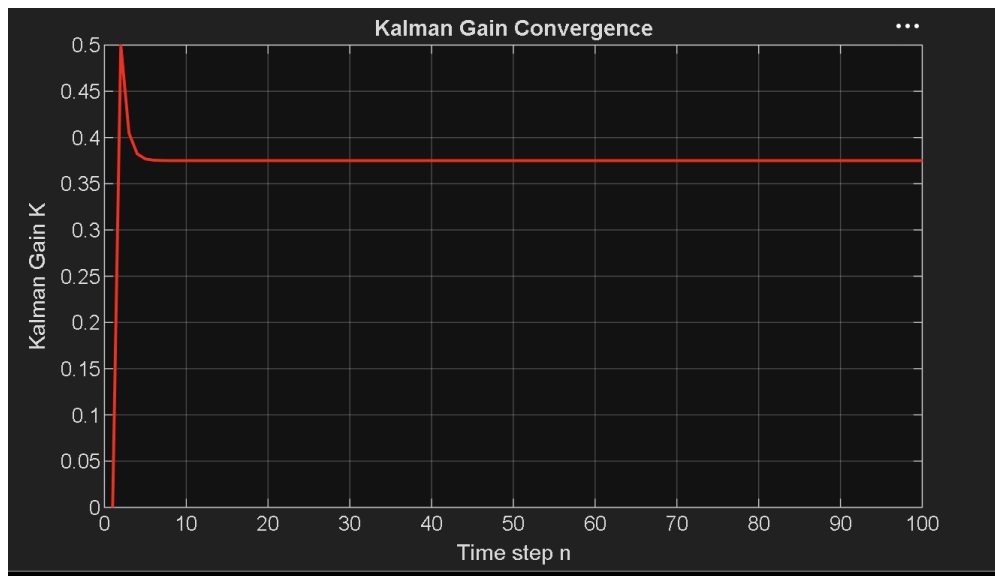


Figure 5: Convergence of the Kalman gain K_n over time.

Inferences from Figure 5 (Kalman Gain Convergence):

- At $n = 1$, the first computed gain is $K_1 \approx 0.5$. This follows from the initialisation $P_0 = Q/(1 - a^2) = 1.0$: the prediction error variance $P_{1|0} = a^2 P_0 + Q = 0.64 + 0.36 = 1.0$ equals the observation noise $R = 1$, so the filter splits trust equally between model and measurement at the very first step.
- K_n drops rapidly and converges to $K_\infty \approx 0.385$ within the first **5–10 steps**, after which it remains essentially constant.
- The steady-state value of 0.385 means the filter assigns 38.5% weight to the new observation and 61.5% to the model prediction at every step. This reflects the fact that $R = 1.0 > Q = 0.36$ — the observation noise is larger than the system noise, so the model is trusted more.

- The fast convergence means the filter reaches its optimal operating mode very quickly, regardless of the initial conditions.
- Once converged, the filter effectively becomes a **fixed first-order recursive smoother**, applying the same weights at every step — a direct consequence of the Riccati equation reaching its fixed point at $P_{\infty}^{-} \approx 0.625$.

3. Problem III.3 — Output Autocovariance of a Digital Filter

3.1 Problem Statement

Generate a sequence of 1024 zero-mean unit-variance Gaussian random variables and pass it through a system with impulse response $h_n = e^{-2n}$ for $n \geq 0$. Estimate the autocovariance of the output process and compare it to the theoretical autocovariance.

3.2 Theoretical Background

Let $\{X_n\}$ be zero-mean, unit-variance white noise: $R_X(k) = \delta(k)$. The output of a discrete-time LTI filter with impulse response $\{h_n\}$ is:

$$Y_n = \sum_{j=0}^{\infty} h_j X_{n-j}. \quad (12)$$

Since the input is WSS, the output is also WSS, with autocorrelation obtained by substituting $R_X(k) = \delta(k)$ into the general LTI input-output formula:

$$R_Y(k) = \sum_{m=0}^{\infty} h_m h_{m+|k|}. \quad (13)$$

By the **Einstein-Wiener-Khinchin theorem**, $S_Y(\omega) = |H(\omega)|^2 S_X(\omega) = |H(\omega)|^2$ (since white noise has flat spectrum $S_X(\omega) = 1$), giving the same result via the spectral route.

Closed-form result. For $h_n = e^{-2n}$:

$$R_Y(k) = \sum_{m=0}^{\infty} e^{-2m} \cdot e^{-2(m+|k|)} = e^{-2|k|} \sum_{m=0}^{\infty} e^{-4m} = \frac{e^{-2|k|}}{1 - e^{-4}}. \quad (14)$$

$$\boxed{R_Y(k) = \frac{e^{-2|k|}}{1 - e^{-4}}} \quad (1/(1 - e^{-4}) \approx 1.0746). \quad (15)$$

3.3 MATLAB Code

Listing 3: MATLAB code for Digital Filter Autocovariance

```

1 clear; clc; close all;
2 % Parameters
3 N      = 1024;
4 max_lag = 30;
5 % Input: white noise
6 X = randn(1, N);
7 % Impulse response h_n = e^(-2n), truncated at n = 20
8 n_h = 0:20;
```

```

9  h      = exp(-2 * n_h);
10 % Output via convolution
11 Y_full = conv(h, X);
12 Y       = Y_full(1:N);
13 % Empirical autocovariance
14 acf_emp = zeros(1, max_lag + 1);
15 for k = 0:max_lag
16     acf_emp(k+1) = mean(Y(k+1:end) .* Y(1:end-k));
17 end
18 % Theoretical autocovariance
19 k_vals      = 0:max_lag;
20 acf_theory = exp(-2 * k_vals) / (1 - exp(-4));
21 % Plot 1: Input vs Output (first 100 samples)
22 figure;
23 plot(X(1:100), 'Color', [0.6 0.6 0.6], 'LineWidth', 1); hold
    on;
24 plot(Y(1:100), 'b', 'LineWidth', 1.5);
25 xlabel('Sample n'); ylabel('Amplitude');
26 title('White Noise Input vs Filter Output (first 100 samples)');
27 legend('Input X (white noise)', 'Output Y (correlated)');
28 grid on;
29 saveas(gcf, 'filter_inout.png');
30 % Plot 2: Empirical vs Theoretical Autocovariance
31 figure;
32 plot(k_vals, acf_emp, 'bo-', 'LineWidth', 1.5, 'MarkerSize',
    5); hold on;
33 plot(k_vals, acf_theory, 'r--', 'LineWidth', 1.8);
34 xlabel('Lag k'); ylabel('R_Y(k)');
35 title('Output Autocovariance - Empirical vs Theoretical');
36 legend('Empirical', 'Theoretical');
37 grid on;
38 saveas(gcf, 'filter_acf.png');
39 % Print values
40 fprintf('R_Y(0) empirical    = %.4f\n', acf_emp(1));
41 fprintf('R_Y(0) theoretical = %.4f\n', acf_theory(1));

```

3.4 Results and Discussion

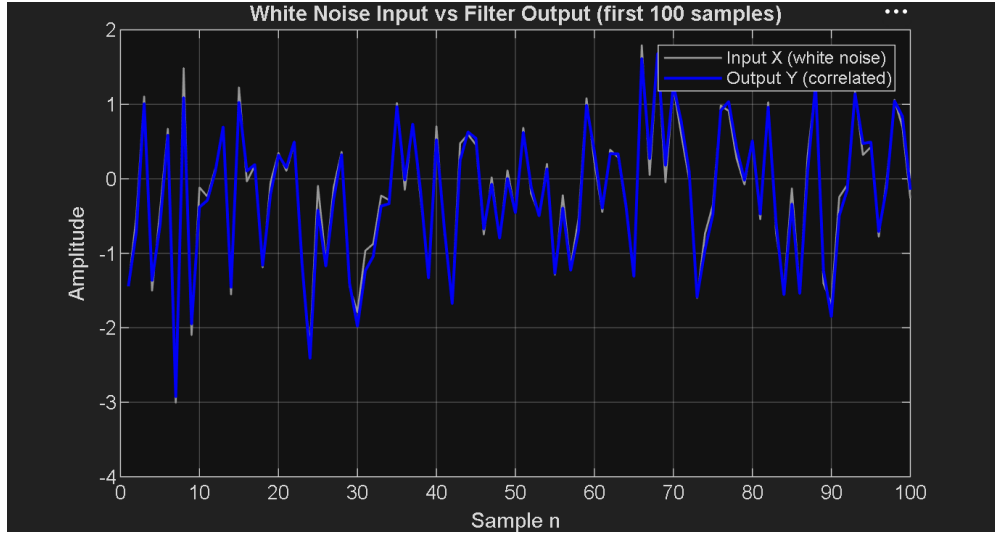


Figure 6: First 100 samples of white noise input X_n (gray) and filter output Y_n (blue).

Inferences from Figure 6 (Input vs. Output):

- The output Y_n (blue) looks visually similar to the white noise input X_n (gray), because $h_n = e^{-2n}$ decays very fast: $h_0 = 1$, $h_1 \approx 0.135$, $h_2 \approx 0.018$. The current input sample dominates each output sample.
- Despite the similarity, the output is **not white noise** — neighbouring Y_n values share common input samples and are correlated. The autocovariance plot reveals this clearly.
- The amplitude range of Y_n is slightly wider than that of X_n . This reflects the filter's power gain: $R_Y(0) \approx 1.075 > R_X(0) = 1$, since the output accumulates weighted contributions from multiple past inputs.

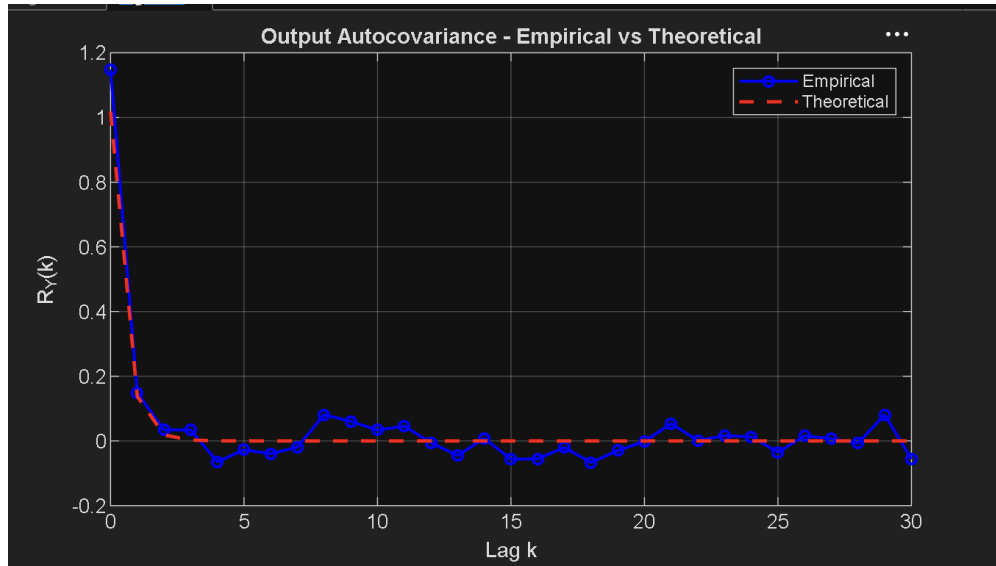


Figure 7: Empirical autocovariance (blue circles) vs. theoretical $R_Y(k) = e^{-2|k|}/(1 - e^{-4})$ (red dashed).

Inferences from Figure 7 (Autocovariance):

- At lag $k = 0$: both empirical and theoretical values agree near $R_Y(0) \approx 1.075$, which exceeds the input variance of 1.0. This is the power gain from the filter — consistent with $S_Y(\omega) = |H(\omega)|^2$ integrating to a value greater than 1.
- For small lags ($k = 1, 2, 3$): the empirical dots closely follow the theoretical exponential curve, confirming the formula $R_Y(k) = e^{-2|k|}/(1 - e^{-4})$.
- The autocovariance decays **exponentially** with lag — in direct contrast to the white noise input $R_X(k) = \delta(k)$ which is zero for all $k \neq 0$. The filter has introduced temporal memory into the output.
- By $k \approx 5$, the theoretical $R_Y(k)$ is essentially zero. The empirical estimates confirm this, scattering randomly around zero for all larger lags.
- The scatter around zero at large lags is **estimation noise** ($\propto 1/\sqrt{N}$), not a failure of theory. It would shrink with a longer input sequence.
- Both derivation routes — the time-domain formula $R_Y(k) = \sum_m h_m h_{m+|k|}$ and the spectral relation $S_Y(\omega) = |H(\omega)|^2 S_X(\omega)$ — produce the same theoretical curve, confirming their consistency.